

Pydantic Workshop / July 2026

Your agent needs a sandbox, not a desert

Samuel Colvin · Pydantic

whoami - Samuel Colvin

- Creator of **Pydantic**
- Founder & CEO of **Pydantic Labs**

The Pydantic Stack

- **Pydantic Validation**: data validation library
- **Pydantic AI**: Agent Framework
- **Pydantic Logfire**: Observability - AI & Infra, Evals, Prompt management etc.

One platform. One trace.

**Replace your separate infra and AI
observability platforms with
Pydantic Logfire.**

Agents & Tools

Agents are easy to define in the abstract.

```
env = Environment()
tools = Tools(env)
system_prompt = "Goals, constraints, and how to act"

while True:
    action = llm.run(system_prompt + env.state)
    env.state = tools.run(action)
```

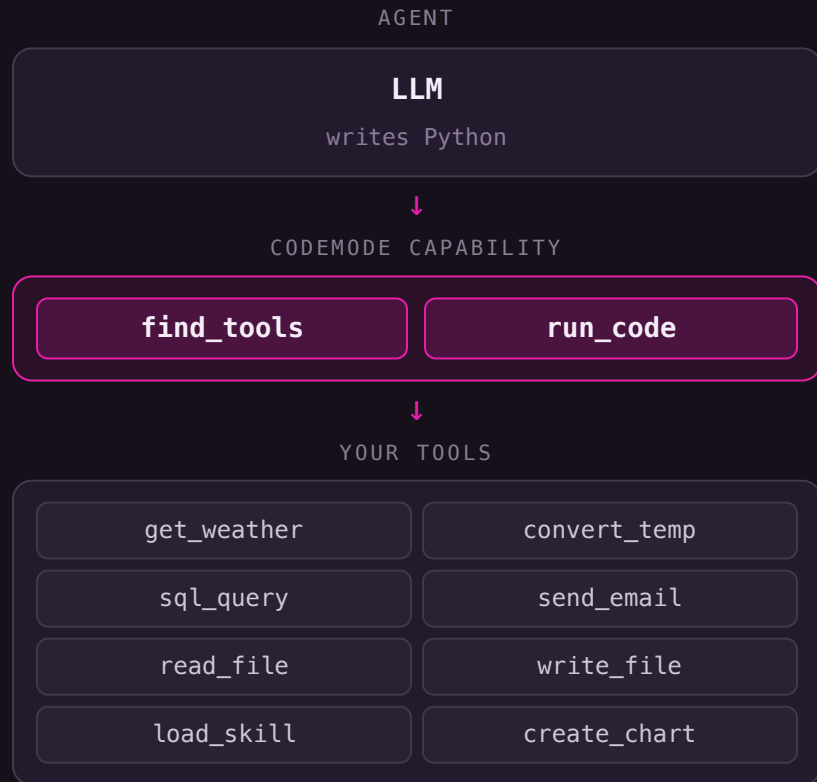
Agents are models using tools in a loop

... but how many tools?

```
{
    "get_weather",
    "search_flights",
    "book_hotel"
}
```

Enter Codemode 🥁

- aka "Programatic tool calling", CodeAct, and RLM (Recursive Language Models)
- The model writes Python (or js); the Python calls tools
- Tool composition, branching, looping - all happen in **code**, not round-trips
- Intermediate state stays in the runtime, not in the context window
- The agent only ever sees **two** tools: `run_code` and `find_tools`
- "1000 tools in 1000 tokens" is the promise
- even better "1000 tool calls with 1 LLM call"



But how do we run it?

The people who benefit most from codemode are those
selling sandboxes.

Two Paradigms

Both alike in dignity.

Light tasks

- Pass tool outputs to next tool
- Calculations
- Run queries
- Render a chart

TYPICAL DURATION

milliseconds

Heavy tasks

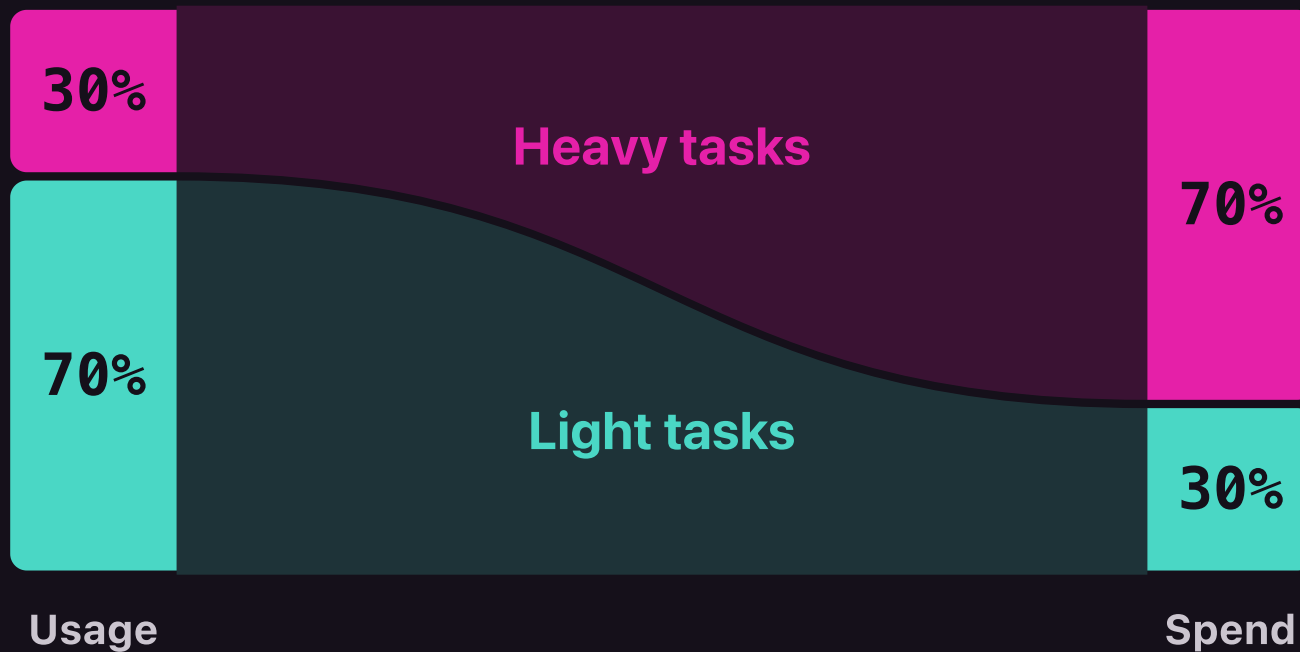
- Clone a repo
- Run a coding agent
- Heavy computation
- Compilation

TYPICAL DURATION

minutes

Two Paradigms

Both alike in dignity. **Usage**



Two Paradigms

Both alike in dignity. **Problems**

Light tasks

Execution environments:

- VM
- Container
- gVisor
- Firecracker

TYPICAL BOOT TIME

1-3s

Heavy tasks

Execution environments:

- VM
- Container
- gVisor
- Firecracker

TYPICAL BOOT TIME

1-3s

The sandbox vs. the desert

THE DESERT

awk sed gcc g++ **clang** **less** more
grep egrep fgrep **make** cmake
bash zsh sh **fish** curl wget
tar gzip **bzip2** xz zip unzip
find **xargs** **locate** vim nano
emacs **ed** python3 pip pipx node
npm pnpm yarn git svn **hg** ssh
scp rsync **cron** **systemd** apt
apt-get dpkg yum **dnf** brew jq yq
tr **cut** sort uniq head tail

THE SANDBOX

sql_query

create_chart

create_table

load_skill

Sandbox, not a desert

- A **curated list** beats the kitchen sink
- Most agents don't need a whole Linux box - they need ~5 tools used well
- "Analyse BI data to find efficiencies" sounds broad - the actual surface is tiny
- Full sandboxes solve a problem most embedded agents don't have

What would a better sandbox for light tasks look like?

Introducing Pydantic Monty

- A **minimal Python interpreter, written in Rust**, for code written by AI
- Safe by default: no filesystem, env, or network unless you grant it
- Pass in your own functions - the model calls them as ordinary Python
- **Type-checks** code (via `ty`) before it runs
- **Snapshottable** - pause at an external call, resume later, anywhere
- Supports a small but growing subsection of the standard library - `re`, `json`, `datetime`, `Pathlib` etc.
- Monty code is run in a pool of subprocesses, wire protocol for communicating with the main process

```
➤ uv add pydantic-monty
➤ npm i @pydantic/monty
➤ cargo add monty
```

The advantages

STARTUP LATENCY

5 μ s
MONTY

VS.

~~1s~~
FULL SANDBOX

SNAPSHOT SIZE

1KB
MONTY

VS.

~~1GB~~
FULL SANDBOX

COST PER RUN

\$0
MONTY

VS.

~~\$?~~
FULL SANDBOX

Demo



Roadmap

- **Next week** — hackmonty.com round 3, **\$20,000** prize
- **The week after** — **Monty V1** release
- **Mid August** — **Monty service** in Logfire (`monty-over-websocket`)
- **Once we have sufficient design partners** — Monty wire protocol for full sandboxes (`python-over-websocket`)

Thank you

contact

sales@pydantic.dev

@samuelcolvin · @pydantic